BFS Cycle Detection (Undirected)

```
1: function HasCycleBFS(V, Adj)
2:     visited[0...V-1] ← false
3:     for each vertex s do
4:         if visited[s] = false then
5:             Create empty queue
6:             visited[s] ← true
7:             Push (s, −1)
8:             while queue not empty do
9:                 (node, parent) ← Pop
10:                for neighbor in Adj[node] do
11:                    if visited[neighbor]=false then
12:                        visited[neighbor]←true
13:                        Push(neighbor, node)
14:                    else if neighbor ≠ parent then
15:                        return True
16:                    end if
17:                end for
18:            end while
19:        end if
20:    end for
21:    return False
22: end function
```

DFS Cycle Detection (Undirected)

```
1: function HasCycleDFS(V, Adj)
2:     visited[0...V-1]←false
3:     for each vertex u do
4:         if visited[u]=false then
5:             if DFS(u, −1) then
6:                 return True
7:             end if
8:         end if
9:     end for
10:    return False
11: end function

12: function DFS(node, parent)
13:     visited[node]←true
14:     for neighbor in Adj[node] do
15:         if visited[neighbor]=false then
16:             if DFS(neighbor, node) then
17:                 return True
18:             end if
19:         else if neighbor≠parent then
20:             return True
21:         end if
22:     end for
23:     return False
24: end function
```

Kahn's Algorithm / BFS Cycle Detection (Directed)

```
1: function HasCycleKahn(V, Adj)
2:     indegree[0...V-1]← 0
3:     for each vertex u do
4:         for neighbor in Adj[u] do
5:             indegree[neighbor]++
6:         end for
7:     end for
8:     Create empty queue
9:     for each vertex u do
10:        if indegree[u]=0 then
11:            Push u
12:        end if
13:    end for
14:    processed← 0
15:    while queue not empty do
16:        node←Pop
17:        processed++
18:        for neighbor in Adj[node] do
19:            indegree[neighbor]–
20:            if indegree[neighbor]=0 then
21:                Push neighbor
22:            end if
23:        end for
24:    end while
25:    if processed==V then
26:        return False
27:    else
28:        return True
29:    end if
30: end function
```

DFS Cycle Detection (Directed)

```
1: function HasCycleDFS(V, Adj)
2:     visited[0...V-1]←false
3:     recStack[0...V-1]←false
4:     for each vertex u do
5:         if visited[u]=false then
6:             if DFS(u) then
7:                 return True
8:             end if
9:         end if
10:    end for
11:    return False
12: end function

13: function DFS(node)
14:     visited[node]←true
15:     recStack[node]←true
16:     for neighbor in Adj[node] do
17:         if visited[neighbor]=false then
18:             if DFS(neighbor) then
19:                 return True
20:             end if
21:         else if recStack[neighbor] then
22:             return True
23:         end if
24:     end for
25:     recStack[node]←false
26:     return False
27: end function
```